HDOC Day-14

Menu-driven program using switch-case

1. Problem Statement

Input a positive integer and perform one operation according to the user's menu choice:

- Choice 1: Check whether all digits of the number are unique.
- Choice 2: Find the frequency of every digit from 0 to 9.

2. Algorithm

1. Start.

2. Input a positive integer n.

3. Display the menu: 1) Check unique digits 2) Find digit frequencies.

4. Input the user's choice.

5. Use switch(choice) to perform the selected operation.

6. For Choice 1: initialize freq[10] to 0; extract each digit using digit = temp % 10; if that digit has already appeared, mark the number as Not Unique; otherwise increment its frequency; continue with temp = temp / 10.

7. For Choice 2: initialize freq[10] to 0; extract every digit and increment freq[digit]; then print freq[0] through freq[9].

8. For an invalid menu choice, display "Invalid choice".

9. Stop.

3. Test Case Table

Test	Operation	Input Number	Choice	Expected Output	Purpose
TC1	Unique digits	12345	1	Unique	All digits are different
TC2	Unique digits	12234	1	Not Unique	Digit 2 repeats
TC3	Digit frequency	112333	2	0:0, 1:2, 2:1, 3:3, 4–9:0	Checks repeated digits
TC4	Digit frequency	50705	2	0:2, 5:2, 7:1; others:0	Checks zero and repeats

4. C Program

```
#include <stdio.h>

int main()
{
    long long n, temp;
```

```
    int choice, digit, i;
    int freq[10] = {0};
    int unique = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    printf("\n1. Check whether all digits are unique");
    printf("\n2. Find frequency of each digit from 0 to 9");
    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            temp = n;
            if (temp == 0)
                freq[0] = 1;

            while (temp > 0)
            {
                digit = temp % 10;
                if (freq[digit] == 1)
                {
                    unique = 0;
                    break;
                }
                freq[digit]++;
                temp = temp / 10;
            }

            if (unique)
                printf("Unique\n");
            else
                printf("Not Unique\n");
            break;

        case 2:
            temp = n;
            if (temp == 0)
                freq[0] = 1;

            while (temp > 0)
            {
                digit = temp % 10;
                freq[digit]++;
                temp = temp / 10;
            }

            for (i = 0; i <= 9; i++)
                printf("freq_%d = %d\n", i, freq[i]);
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}
```

5. Sample Execution

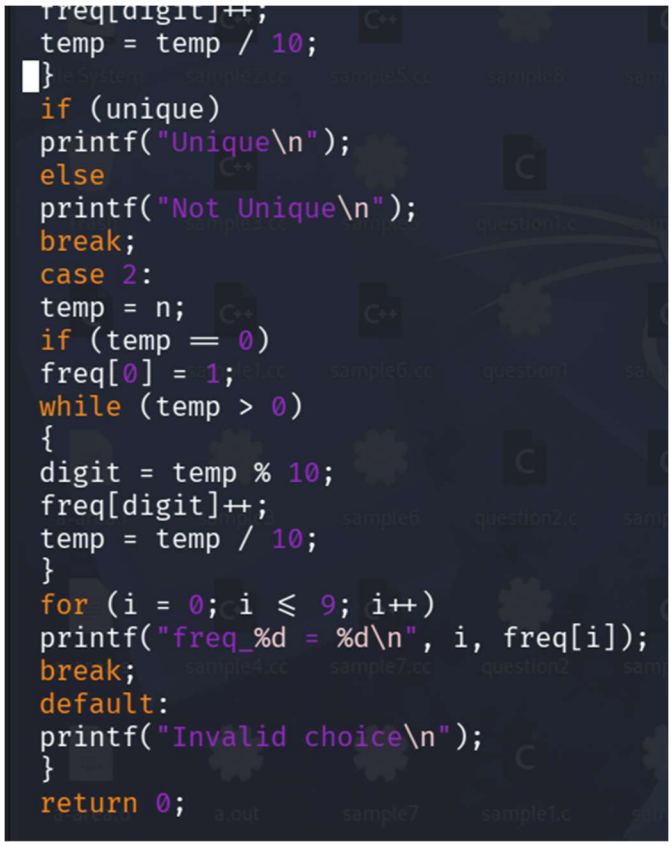
```
Example 1
Enter a positive integer: 12345
1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 1
Unique

Example 2
Enter a positive integer: 112333
1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 2
freq_0 = 0
freq_1 = 2
freq_2 = 1
freq_3 = 3
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0
```

6. Screenshots

Paste your own screenshots after compiling and running the program. Recommended screenshots are provided below.

Screenshot 1 — Source Code / Compilation

```c
#include <stdio.h>
int main()
{
 long long n, temp;
 int choice, digit, i;
int freq[10] = {0};
 int unique = 1;
 printf("Enter a positive integer: ");
 scanf("%lld", &n);
 printf("\n1. Check whether all digits are unique");
 printf("\n2. Find frequency of each digit from 0 to 9");
 printf("\nEnter your choice: ");
 scanf("%d", &choice);
 switch (choice)
 {
 case 1:
 temp = n;
 if (temp == 0)
 freq[0] = 1;
 while (temp > 0)
 {
 digit = temp % 10;
 if (freq[digit] == 1)
 {
 unique = 0;
 break;
 }
 freq[digit]++;
```



```c
 temp = temp / 10;
 }
 if (unique)
 printf("Unique\n");
 else
 printf("Not Unique\n");
 break;
 case 2:
 temp = n;
 if (temp == 0)
 freq[0] = 1;
 while (temp > 0)
 {
 digit = temp % 10;
 freq[digit]++;
 temp = temp / 10;
 }
 for (i = 0; i <= 9; i++)
 printf("freq_%d = %d\n", i, freq[i]);
 break;
 default:
 printf("Invalid choice\n");
 }
 return 0;
```

Screenshot 2 — Choice 1 Output (Unique / Not Unique)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./sample10
Enter a positive integer: 12345

1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 1
Unique
```

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./sample10
Enter a positive integer: 12234

1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 1
Not Unique
```

Screenshots (continued)

Screenshot 3 — Choice 2 Output (Digit Frequencies)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./sample10
Enter a positive integer: 112333

1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 2
freq_0 = 0
freq_1 = 2
freq_2 = 1
freq_3 = 3
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0
```

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./sample10
Enter a positive integer: 50705

1. Check whether all digits are unique
2. Find frequency of each digit from 0 to 9
Enter your choice: 2
freq_0 = 2
freq_1 = 0
freq_2 = 0
freq_3 = 0
freq_4 = 0
freq_5 = 2
freq_6 = 0
freq_7 = 1
freq_8 = 0
freq_9 = 0
```